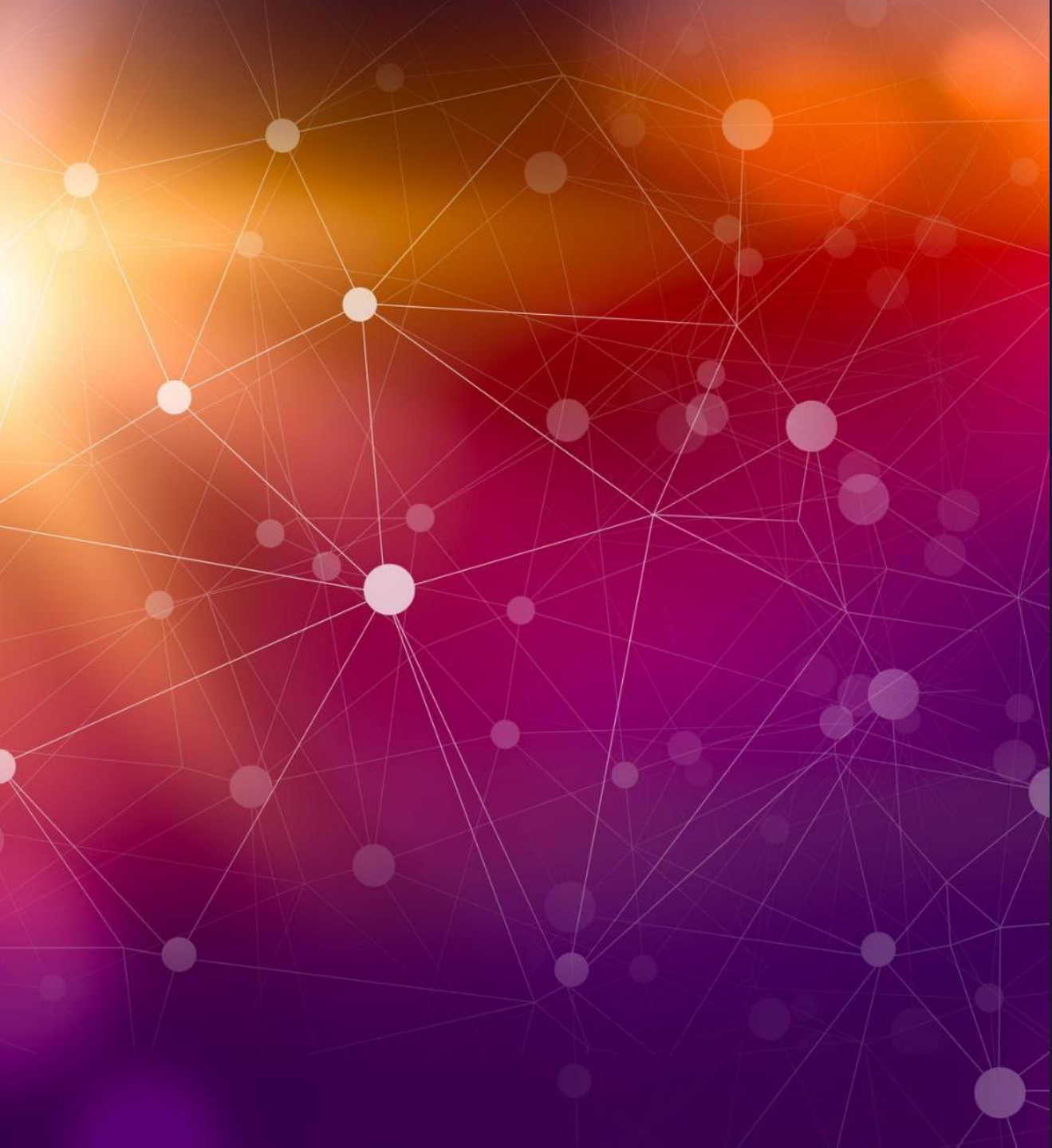




CPU Scheduling Algorithms Analysis

Author: Roy Lozano

*CSCI 4103 (Operating Systems)
Instructor: Dr. Ahmed Sleman*

First Come First Serve (FCFS) Algorithm

STEP 1: Sort processes by arrival time to determine execution order to abide by first come first serve

STEP 2: Execute each process in the order that it arrived

STEP 3: If CPU is idle, advance to process arrival

STEP 4: Calculate response time (time from arrival to first CPU allocation).

STEP 5: Record the process execution time in Gantt chart

STEP 6: Execute the process completely (non-preemptive)

STEP 7: Calculate performance metrics

STEP 8: Add final timeline marker

```
30 def fcfs(self):
31     """First-Come, First-Served Scheduling"""
32     self.reset_processes()
33     # Sort by arrival time
34     sorted_processes = sorted(self.processes, key=lambda x: x.arrival_time)
35
36     current_time = 0
37     for process in sorted_processes:
38         if current_time < process.arrival_time:
39             current_time = process.arrival_time
40
41         # Response time is when process first gets CPU
42         process.response_time = current_time - process.arrival_time
43
44         # Add to Gantt chart
45         self.gantt_chart.append(f"P{process.pid}")
46         self.timeline.append(current_time)
47
48         current_time += process.burst_time
49         process.completion_time = current_time
50         process.turnaround_time = process.completion_time - process.arrival_time
51         process.waiting_time = process.turnaround_time - process.burst_time
52
53     self.timeline.append(current_time)
```

FCFS SUMMARY: Non-preemptive algorithm that selects processes based on priority values. Lower priority number means higher priority. Highest priority (lowest number) gets CPU first. Can cause starvation of low-priority processes. Common solution: Priority aging (gradually increase priority of waiting processes). Used in real-time systems where certain tasks must complete first

Shortest Process Next (SPN) Algorithm

STEP 1: Continue until all processes are completed

STEP 2: Add all newly arrived processes to ready queue and check each process to see if it has arrived by current time

STEP 3: Select and execute process if ready queue is not empty

STEP 4: Apply SPN selection criteria and choose shortest burst time. This is the core of the algorithm: shortest job gets priority

STEP 5: Calculate response time (first time getting CPU)

STEP 6: Record execution in Gantt chart

STEP 7: Execute process to completion (non-preemptive)

STEP 8: Calculate performance metrics

```
55 def spn(self):
56     """Shortest Process Next (Non-preemptive)"""
57     self.reset_processes()
58     ready_queue = []
59     completed = []
60     current_time = 0
61
62     while len(completed) < len(self.processes):
63         # Add arriving processes to ready queue
64         for process in self.processes:
65             if (process.arrival_time <= current_time and
66                 process not in ready_queue and process not in completed):
67                 ready_queue.append(process)
68
69         if ready_queue:
70             # Select process with shortest burst time
71             current_process = min(ready_queue, key=lambda x: x.burst_time)
72             ready_queue.remove(current_process)
73
74             # Set response time if first time running
75             if current_process.response_time == -1:
76                 current_process.response_time = current_time - current_process.arrival_time
77
78             # Add to Gantt chart
79             self.gantt_chart.append(f"P{current_process.pid}")
80             self.timeline.append(current_time)
81
82             current_time += current_process.burst_time
83             current_process.completion_time = current_time
84             current_process.turnaround_time = current_process.completion_time - current_process.arrival_time
85             current_process.waiting_time = current_process.turnaround_time - current_process.burst_time
86             completed.append(current_process)
87         else:
88             current_time += 1
89
90     self.timeline.append(current_time)
```

SPN SUMMARY: Non-preemptive algorithm that selects the process with shortest burst time. Also known as Shortest Job First (SJF). Minimizes average waiting time but can cause starvation of long processes. Requires knowledge of process execution times in advance. Optimal for minimizing average completion time

Shortest Remaining Time (SRT) Algorithm

STEP 1: Continue until all processes complete

STEP 2: Add newly arrived processes to ready queue

STEP 3: Process scheduling decision if ready queue has processes

STEP 4: Apply SRT selection, find process with shortest remaining time. This is checked every time unit, enabling preemption

STEP 5: Check for preemption - switch if different process is selected

STEP 6: Set response time for first execution

STEP 7: Execute current process for 1 time unit

STEP 8: Check if current process completed

STEP 9: Handle CPU idle time

STEP 10: Ensure timeline ends properly

```
92 def srt(self):
93     """Shortest Remaining Time (Preemptive)"""
94     self.reset_processes()
95     ready_queue = []
96     completed = []
97     current_time = 0
98     current_process = None
99
100     while len(completed) < len(self.processes):
101         # Add arriving processes to ready queue
102         for process in self.processes:
103             if (process.arrival_time <= current_time and
104                 process not in ready_queue and process not in completed):
105                 ready_queue.append(process)
106
107         if ready_queue:
108             # Select process with shortest remaining time
109             shortest_process = min(ready_queue, key=lambda x: x.remaining_time)
110
111             # Preemption check
112             if current_process != shortest_process:
113                 if current_process:
114                     self.timeline.append(current_time)
115                     current_process = shortest_process
116                     self.gantt_chart.append(f"P({current_process.pid})")
117                     self.timeline.append(current_time)
118
119                 # Set response time if first time running
120                 if current_process.response_time == -1:
121                     current_process.response_time = current_time - current_process.arrival_time
122
123             # Execute for 1 time unit
124             current_process.remaining_time -= 1
125             current_time += 1
126
127             # Check if process completed
128             if current_process.remaining_time == 0:
129                 current_process.completion_time = current_time
130                 current_process.turnaround_time = current_process.completion_time - current_process.arrival_time
131                 current_process.waiting_time = current_process.turnaround_time - current_process.burst_time
132                 completed.append(current_process)
133                 ready_queue.remove(current_process)
134                 current_process = None
135             else:
136                 current_time += 1
137
138         if self.timeline and self.timeline[-1] != current_time:
139             self.timeline.append(current_time)
```

SRT SUMMARY: Preemptive version of Shortest Process Next (SPN). Always executes the process with the shortest remaining execution time. Can interrupt currently running process if a shorter job arrives. Provides optimal average Completion time but high context switching overhead. May cause starvation of longer processes. Also known as Shortest Remaining Time First (SRTF)

Round Robin (RR) Algorithm

STEP 1: Continue until all processes complete

STEP 2: Add newly arrived processes to end of ready queue # This maintains the FIFO order for fair scheduling

STEP 3: Execute next process in queue if available

STEP 4: Get next process from front of queue (FIFO order)

STEP 5: Set response time on first execution

STEP 6: Record process execution in Gantt chart

STEP 7: Execute for quantum time or remaining time (whichever is smaller). This is the key feature: time slice limitation

STEP 8: Handle process completion or re-queuing

STEP 9: Handle CPU idle time

```
141 def round_robin(self, quantum=2):
142     """Round Robin Scheduling"""
143     self.reset_processes()
144     ready_queue = []
145     completed = []
146     current_time = 0
147
148     while len(completed) < len(self.processes):
149         # Add arriving processes to ready queue
150         for process in self.processes:
151             if (process.arrival_time <= current_time and
152                 process not in ready_queue and process not in completed):
153                 ready_queue.append(process)
154
155         if ready_queue:
156             current_process = ready_queue.pop(0)
157
158             # Set response time if first time running
159             if current_process.response_time == -1:
160                 current_process.response_time = current_time - current_process.arrival_time
161
162             # Add to Gantt chart
163             self.gantt_chart.append(f"P{current_process.pid}")
164             self.timeline.append(current_time)
165
166             # Execute for quantum time or remaining time
167             execution_time = min(quantum, current_process.remaining_time)
168             current_process.remaining_time -= execution_time
169             current_time += execution_time
170
171             # Check if process completed
172             if current_process.remaining_time == 0:
173                 current_process.completion_time = current_time
174                 current_process.turnaround_time = current_process.completion_time - current_process.arrival_time
175                 current_process.waiting_time = current_process.turnaround_time - current_process.burst_time
176                 completed.append(current_process)
177             else:
178                 # Add back to ready queue if not completed
179                 ready_queue.append(current_process)
180
181             # Add back to ready queue if not completed
182             else:
183                 current_time += 1
184
185         self.timeline.append(current_time)
```

*RR SUMMARY: Preemptive algorithm that gives each process a fixed time slice (quantum). Processes are scheduled in circular queue fashion (FIFO with time limits). Fair scheduling: no process waits indefinitely (no starvation). Performance depends heavily on quantum size: * Small quantum: Good response time, high context switching overhead. * Large quantum: resembles FCFS behavior - Ideal for interactive systems and time-sharing environments*

Priority Scheduling (PRI) Algorithm

STEP 1: Continue until all processes complete

STEP 2: Add newly arrived processes to ready queue

STEP 3: Select and execute process if ready queue has processes

STEP 4: Apply priority selection criteria Select process with highest priority (lowest priority number)

STEP 5: Set response time for first execution

STEP 6: Record process execution in Gantt chart

STEP 7: Execute process to completion (non-preemptive). Once a process starts, it runs until completion regardless of higher priority processes that may arrive later

STEP 8: Calculate performance metrics

STEP 9: Handle CPU idle time when no processes are ready

```
185 def priority_scheduling(self):
186     """Priority Scheduling (Non-preemptive) - Lower number = Higher priority"""
187     self.reset_processes()
188     ready_queue = []
189     completed = []
190     current_time = 0
191
192     while len(completed) < len(self.processes):
193         # Add arriving processes to ready queue
194         for process in self.processes:
195             if (process.arrival_time <= current_time and
196                 process not in ready_queue and process not in completed):
197                 ready_queue.append(process)
198
199         if ready_queue:
200             # Select process with highest priority (lowest number)
201             current_process = min(ready_queue, key=lambda x: x.priority)
202             ready_queue.remove(current_process)
203
204             # Set response time if first time running
205             if current_process.response_time == -1:
206                 current_process.response_time = current_time - current_process.arrival_time
207
208             # Add to Gantt chart
209             self.gantt_chart.append(f"P{current_process.pid}")
210             self.timeline.append(current_time)
211
212             current_time += current_process.burst_time
213             current_process.completion_time = current_time
214             current_process.turnaround_time = current_process.completion_time - current_process.arrival_time
215             current_process.waiting_time = current_process.turnaround_time - current_process.burst_time
216             completed.append(current_process)
217         else:
218             current_time += 1
219
220     self.timeline.append(current_time)
```

PRIORITY SCHEDULING SUMMARY: Non-preemptive algorithm that selects processes based on priority value. Lower priority number means higher priority (0 is highest priority). Process with highest priority gets CPU first. Can cause indefinite blocking (starvation) of low-priority processes. Common solution: Priority aging (gradually increase priority of waiting processes). Used in real-time systems where certain tasks must complete first

Test Case 1: Mixed Workload

Purpose: General testing with varied arrival times and burst times

Process #	Arrival Time	Burst Time	Priority
1	0	6	3
2	2	8	1
3	4	7	2
4	6	3	4
5	8	4	2

Algorithm	Gantt Chart	Performance
FCFS	0 P1 6 P2 14 P3 21 P4 24 P5 28	Average Waiting Time: 9.00 Average Completion Time: 14.60 Average Response Time: 9.00
SPN	0 P1 6 P4 9 P5 13 P3 20 P2 28	Average Waiting Time: 5.60 Average Completion Time: 11.20 Average Response Time: 5.60
SRT	0 P1 6 P4 9 P5 13 P3 20 P2 28	Average Waiting Time: 5.60 Average Completion Time: 11.20 Average Response Time: 5.60
Round Robin	0 P1 3 P1 6 P2 9 P3 12 P4 15 P2 18 P5 21 P3 24 P2 26 P5 27 P3 28	Average Waiting Time: 10.80 Average Completion Time: 16.40 Average Response Time: 5.00
Priority	0 P1 6 P2 14 P3 21 P5 25 P4 28	Average Waiting Time: 9.20 Average Completion Time: 14.80 Average Response Time: 9.20

Test Case 2: Simultaneous Arrivals

Purpose: Test algorithm behavior when all processes arrive at the same time

Process #	Arrival Time	Burst Time	Priority
1	0	10	2
2	0	1	1
3	0	2	3
4	0	5	1

Algorithm	Gantt Chart	Performance
FCFS	P1 P2 P3 P4 0 10 11 13 18	Average Waiting Time: 8.50 Average Completion Time: 13.00 Average Response Time: 8.50
SPN	P2 P3 P4 P1 0 1 3 8 18	Average Waiting Time: 3.00 Average Completion Time: 7.50 Average Response Time: 3.00
SRT	P2 P3 P4 P1 0 1 3 8 18	Average Waiting Time: 3.00 Average Completion Time: 7.50 Average Response Time: 3.00
Round Robin	P1 P2 P3 P4 P1 P4 P1 P1 0 3 4 6 9 12 14 17 18	Average Waiting Time: 6.00 Average Completion Time: 10.50 Average Response Time: 3.25
Priority	P2 P4 P1 P3 0 1 6 16 18	Average Waiting Time: 5.75 Average Completion Time: 10.25 Average Response Time: 5.75

Test Case 3: Equal Burst Times

Purpose: Test priority and arrival time handling when burst times are identical

Process #	Arrival Time	Burst Time	Priority
1	0	5	4
2	1	5	2
3	2	5	1
4	3	5	3

Algorithm	Gantt Chart	Performance
FCFS	Timeline: 0 5 10 15 20 Events: P1 P2 P3 P4	Average Waiting Time: 6.00 Average Completion Time: 11.00 Average Response Time: 6.00
SPN	Timeline: 0 5 10 15 20 Events: P1 P2 P3 P4	Average Waiting Time: 6.00 Average Completion Time: 11.00 Average Response Time: 6.00
SRT	Timeline: 0 5 10 15 20 Events: P1 P2 P3 P4	Average Waiting Time: 6.00 Average Completion Time: 11.00 Average Response Time: 6.00
Round Robin	Timeline: 0 3 5 8 11 14 16 18 20 Events: P1 P1 P2 P3 P4 P2 P3 P4	Average Waiting Time: 8.25 Average Completion Time: 13.25 Average Response Time: 4.50
Priority	Timeline: 0 5 10 15 20 Events: P1 P3 P2 P4	Average Waiting Time: 6.00 Average Completion Time: 11.00 Average Response Time: 6.00

Test Case 4: High Priority Variation

Purpose: Test priority scheduling with clear priority differences

Process #	Arrival Time	Burst Time	Priority
1	0	4	5
2	1	3	1
3	2	6	3
4	3	2	1
5	4	8	2
6	5	1	4

Algorithm	Gantt Chart	Performance
FCFS	P1 P2 P3 P4 P5 P6 0 4 7 13 15 23 24	Average Waiting Time: 7.83 Average Completion Time: 11.83 Average Response Time: 7.83
SPN	P1 P4 P6 P2 P3 P5 0 4 6 7 10 16 24	Average Waiting Time: 4.67 Average Completion Time: 8.67 Average Response Time: 4.67
SRT	P1 P4 P6 P2 P3 P5 0 4 6 7 10 16 24	Average Waiting Time: 4.67 Average Completion Time: 8.67 Average Response Time: 4.67
Round Robin	P1 P1 P2 P3 P4 P5 P6 P3 P5 P5 0 3 4 7 10 12 15 16 19 22 24	Average Waiting Time: 7.17 Average Completion Time: 11.17 Average Response Time: 5.50
Priority	P1 P2 P4 P5 P3 P6 0 4 7 9 17 23 24	Average Waiting Time: 7.50 Average Completion Time: 11.50 Average Response Time: 7.50

Test Case 5: Large Burst Times

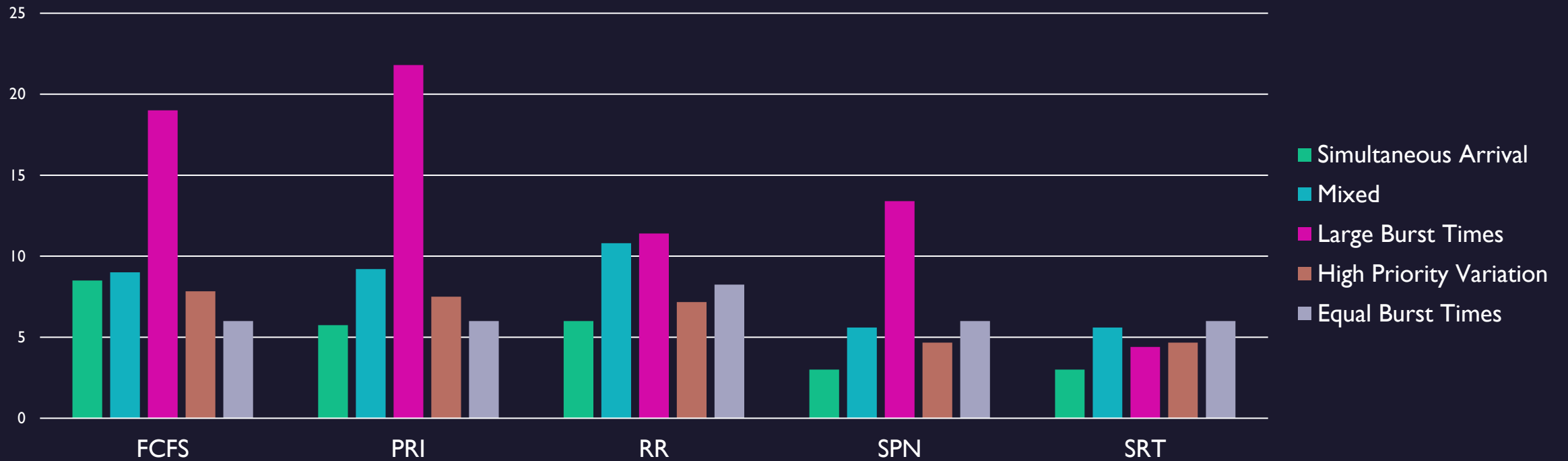
Purpose: Test algorithms with dramatically different execution times

Process #	Arrival Time	Burst Time	Priority
1	0	20	3
2	2	1	2
3	4	15	1
4	6	2	4
5	8	1	5

Algorithm	Gantt Chart	Performance
FCFS	P1 P2 P3 P4 P5 0 20 21 36 38 39	Average Waiting Time: 19.00 Average Completion Time: 26.80 Average Response Time: 19.00
SPN	P1 P2 P5 P4 P3 0 20 21 22 24 39	Average Waiting Time: 13.40 Average Completion Time: 21.20 Average Response Time: 13.40
SRT	P1 P2 P1 P3 P4 P5 P3 P1 0 2 3 4 6 8 9 22 39	Average Waiting Time: 4.40 Average Completion Time: 12.20 Average Response Time: 0.00
Round Robin	P1 P1 P2 P1 P3 P4 P1 P5 P3 P1 P3 P1 P3 P1 P3 0 3 6 7 10 13 15 18 19 22 25 28 31 34 36 39	Average Waiting Time: 11.40 Average Completion Time: 19.20 Average Response Time: 5.40
Priority	P1 P3 P2 P4 P5 0 20 35 36 38 39	Average Waiting Time: 21.80 Average Completion Time: 29.60 Average Response Time: 21.80

Algorithm Wait Time Performance

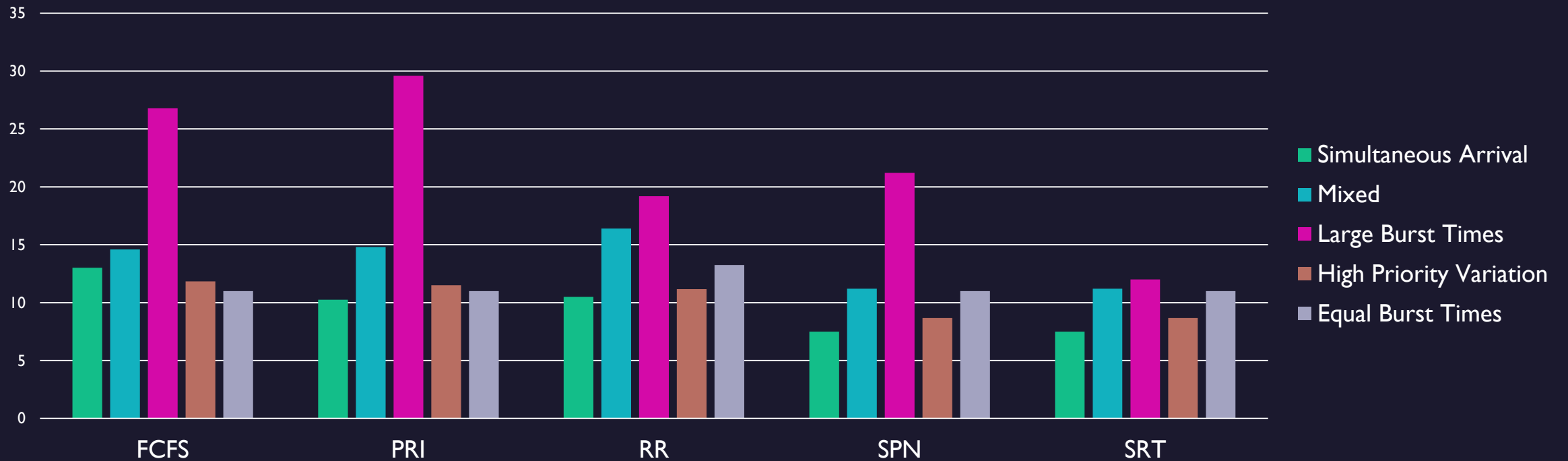
Purpose: Comparison of algorithm wait times across test cases



Wait Time Performance Analysis: Shows SPN/SRT consistently achieving the lowest wait times (3.0-13.4 range), while Round Robin suffers from context-switching overhead (6.0-11.4 range). The analysis shows how job-length awareness dominates performance.

Algorithm Turnaround Time Performance

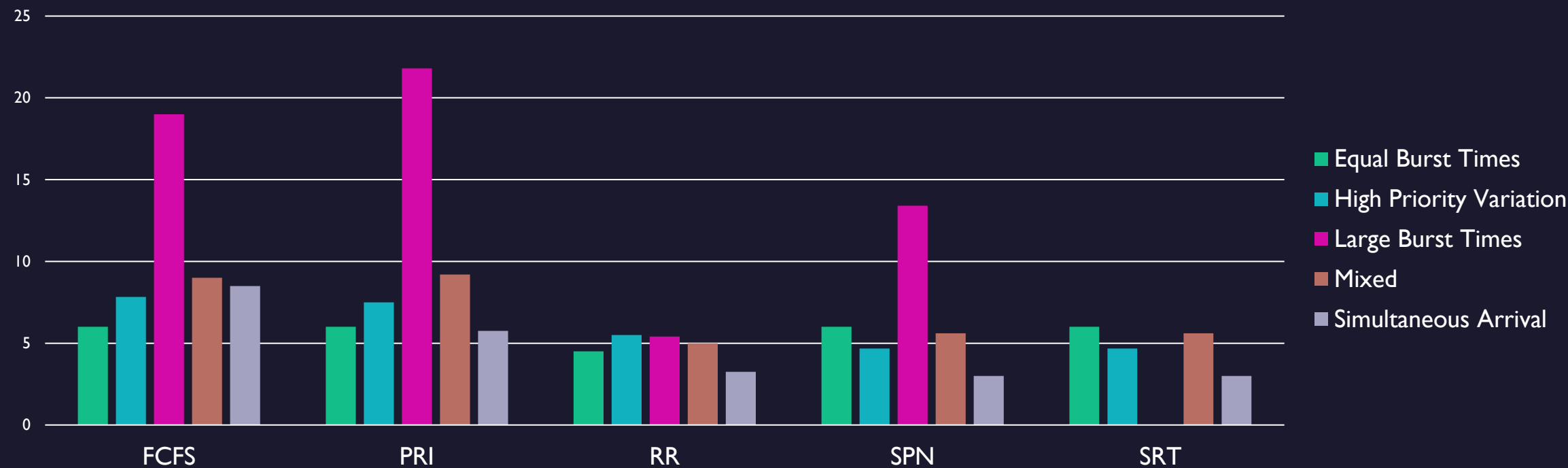
Purpose: Comparison of algorithm completion times across test cases



Turnaround Time Performance Analysis: Confirms SPN/SRT excellence for throughput optimization, with SRT showing dramatic improvement over SPN in the Large Burst Times scenario (12.0 vs 21.2). Reveals why these algorithms are ideal for batch processing.

Algorithm Response Time Performance

Purpose: Comparison of algorithm response times across test cases



Response Time Performance Analysis: Unveils the most significant finding: **SRT achieved 0.0 average response time** in extreme scenarios, exemplifying that perfect responsiveness is theoretically achievable. Shows Round Robin's consistent responsiveness advantage.

Performance Results

None of the given algorithms performed consistently better than the others. Each has its own strengths and weaknesses however there are a few scenarios worth noting.

Throughput Winner: SPN & SRT

Strengths:

- Consistently lowest wait times (3.0-13.4)
- Best turnaround times across scenarios (7.5-21.2 range)

Weaknesses:

- Requires accurate burst time estimation
- Potential for starvation of long processes
- No inherent responsiveness advantage (except SRT in extreme cases)

Best Use Cases:

- Batch processing systems
- Scientific computing workloads
- Systems with predictable job patterns

Responsiveness Winner: Round Robin

Strengths:

- Most consistent response times (3.25-5.5 range)
- No process starvation

Weaknesses:

- Highest wait times in most scenarios (6.0-11.4 range)
- Context switching impacts throughput
- Performance dependent on quantum size

Best Use Cases:

- Interactive computing systems
- Time-sharing environments
- Systems prioritizing user experience

Inconsistent Performer: Priority Scheduling

Strengths:

- Good performance with proper priority assignment
- Supports real-time and system process requirements

Weaknesses:

- Volatile performance (5.75-21.8 wait time)
- Risk of process starvation

Best Use Cases:

- Real-time systems with defined priority requirements
- Applications requiring specific service guarantees

Performance Results Continued

None of the given algorithms performed consistently better than the others. Each has its own strengths and weaknesses however there are a few scenarios worth noting.

Note: SRT's Perfect Response Time

SRT achieved 0.0 average response time in the Large Burst Times scenario.

Perfect Responsiveness: Short processes get immediate CPU access

Preemption Long processes yield instantly to shorter ones

Practical Implication: Some processes can achieve no delay in process handoff

Mixed Workload (Average Test Case)

Performance Ranks:

- **SPN/SRT:** 5.6 wait, 11.2 turnaround, 5.6 response. Provide best overall performance
- **FCFS:** 9.0 wait, 14.6 turnaround, 9.0 response
- **Priority:** 9.2 wait, 14.8 turnaround, 9.2 response
- **Round Robin:** 10.8 wait, 16.4 turnaround, 5.0 response. Best responsiveness at the cost of throughput.

Large Burst Times (Stress Test)

- **SRT's Dominance:** 4.4 wait, 12.0 turnaround, 0.0 response
- **Priority's Failure:** Worst performance across all metrics
- **Preemption's Value:** SRT beats SPN by 204% in wait time

Choosing An Algorithm

Purpose: Summarize the best use cases for each algorithm

System Goal:

Throughput Priority:

- Choose SPN/SRT algorithms

Responsiveness Priority:

- Choose Round Robin (RR) or SRT algorithms

Fairness Priority:

- Choose Round Robin (RR) algorithm

Simplicity Priority:

- Choose First-Come First-Served (FCFS) algorithm

Workload Type:

Uniform Job Sizes:

- Any algorithm will perform adequately

Mixed Job Lengths:

- SPN or SRT algorithms strongly recommended

Unknown/Unpredictable Patterns:

- Round Robin (RR) for safe and consistent performance

Real-time Requirements:

- Priority scheduling with preemption capabilities

Closing Thoughts:

- Modern operating systems use more complex queue types which makes choosing a single algorithm unrealistic
- Different workloads and system goals call for different types of algorithms
- Algorithm performance in one category often has a negative impact in other categories

Data

Test Set	Algorithm	Avg Wait	Avg Turnaround	Avg Response
Mixed	FCFS	9	14.6	9
Mixed	SPN	5.6	11.2	5.6
Mixed	SRT	5.6	11.2	5.6
Mixed	RR	10.8	16.4	5
Mixed	PRI	9.2	14.8	9.2
Simultaneous Arrival	FCFS	8.5	13	8.5
Simultaneous Arrival	SPN	3	7.5	3
Simultaneous Arrival	SRT	3	7.5	3
Simultaneous Arrival	RR	6	10.5	3.25
Simultaneous Arrival	PRI	5.75	10.25	5.75
Equal Burst Times	FCFS	6	11	6
Equal Burst Times	SPN	6	11	6
Equal Burst Times	SRT	6	11	6
Equal Burst Times	RR	8.25	13.25	4.5
Equal Burst Times	PRI	6	11	6
High Priority Variation	FCFS	7.83	11.83	7.83
High Priority Variation	SPN	4.67	8.67	4.67
High Priority Variation	SRT	4.67	8.67	4.67
High Priority Variation	RR	7.17	11.17	5.5
High Priority Variation	PRI	7.5	11.5	7.5
Large Burst Times	FCFS	19	26.8	19
Large Burst Times	SPN	13.4	21.2	13.4
Large Burst Times	SRT	4.4	12	0
Large Burst Times	RR	11.4	19.2	5.4
Large Burst Times	PRI	21.8	29.6	21.8